

Arithmetic Coding of a 6×6 Data Matrix

1. Problem Statement:

The goal of this assignment is to encode a 6×6 matrix of integer data using arithmetic coding. Arithmetic coding is a lossless data compression technique that represents a sequence of symbols as a single floating-point number between 0 and 1.

2. Input Data: The input is a 6×6 matrix of integers:

$$\begin{bmatrix} 52 & 55 & 61 & 66 & 70 & 61 \\ 63 & 59 & 55 & 90 & 109 & 85 \\ 62 & 59 & 68 & 113 & 144 & 104 \\ 63 & 58 & 71 & 122 & 154 & 106 \\ 67 & 61 & 68 & 104 & 126 & 88 \\ 68 & 65 & 70 & 106 & 122 & 90 \end{bmatrix}$$

Each value represents a symbol to be encoded.

3. Steps of Arithmetic Coding:

Step 1: Flatten the Data

Arithmetic coding requires the data as a sequence of symbols, not a 2D array.

```
data_lst = []
for i in range(6):
    for j in range(6):
        data_lst.append(int(data[i][j]))
```

After flattening, data_lst contains 36 elements.

Step 2: Calculating Symbol Probabilities

First, we need to count the occurrences of each symbol:

```
count_dict = {}
for i in data_lst:
    count_dict[i] = count_dict.get(i, 0) + 1
```

Then, we need to calculate the probability of each symbol:

```
probability_dict = {}
for i, j in count_dict.items():
    probability_dict[i] = j / total
```

Suppose the first 7 unique symbols sorted in ascending order are: 52, 55, 58, 59, 61, 62, 63

Symbol	Count	Probability (Count / Total 36)
52	1	$1 / 36 \approx 0.0278$
55	2	$2 / 36 \approx 0.0556$
58	1	$1 / 36 \approx 0.0278$
59	2	$2 / 36 \approx 0.0556$
61	3	$3 / 36 \approx 0.0833$
62	1	$1 / 36 \approx 0.0278$
63	2	$2 / 36 \approx 0.0556$

Step 3: Define Symbol Intervals

Arithmetic coding assigns intervals to each symbol based on their probability:

```
interval_dict = {}
low = 0
for i in sorted(probability_dict):
    high = low + probability_dict[i]
    interval_dict[i] = (low, high)
    low = high
```

For example: Intervals are assigned cumulatively starting from 0.

Symbol	Probability	Low	High
52	0.0278	0.0000	0.0278
55	0.0556	0.0278	0.0834
58	0.0278	0.0834	0.1112
59	0.0556	0.1112	0.1668
61	0.0833	0.1668	0.2501
62	0.0278	0.2501	0.2779
63	0.0556	0.2779	0.3335

Explanation of calculation for each symbol:

- 52: Low = 0, High = $0 + 0.0278 = 0.0278$
- 55: Low = previous High = 0.0278, High = $0.0278 + 0.0556 = 0.0834$
- 58: Low = 0.0834, High = $0.0834 + 0.0278 = 0.1112$

- 59: Low = 0.1112, High = 0.1112 + 0.0556 = 0.1668
- 61: Low = 0.1668, High = 0.1668 + 0.0833 = 0.2501
- 62: Low = 0.2501, High = 0.2501 + 0.0278 = 0.2779
- 63: Low = 0.2779, High = 0.2779 + 0.0556 = 0.3335

This method continues for all remaining symbols in the data.

Step 4: Narrowing the Interval

I started by initializing the main interval to cover the whole range from 0 to 1:

```
low = 0
high = 1
```

Then, for each symbol in my data list, I updated the interval according to the symbol's predefined range. The formula I used was:

```
for i in data_lst:
    l, h = interval_dict[i]
    r = high - low
    high = low + r*h
    low = low + r*l
```

Step 5: Finding the Encoded Value

After narrowing the interval through all symbols, I took the midpoint of the final interval to get my encoded value:

```
encoded_value = (low + high) / 2
print(encoded_value)
```

For example, if my final interval was [0.0010647, 0.0010648], then my encoded value became:

$\text{encoded_value} = (0.0010647 + 0.0010648) / 2 \approx 0.00106475$

This single number now represents my entire 36-symbol sequence.

4. Source Code:

```
import numpy as np

data = np.array([
    [52, 55, 61, 66, 70, 61],
    [63, 59, 55, 90, 109, 85],
    [62, 59, 68, 113, 144, 104],
    [63, 58, 71, 122, 154, 106],
    [67, 61, 68, 104, 126, 88],
```

```

[68, 65, 70, 106, 122, 90]
])

data_lst = []
for i in range(6):
    for j in range(6):
        data_lst.append(int(data[i][j]))
total = len(data_lst)

count_dict = {}
for i in data_lst:
    count_dict[i] = count_dict.get(i, 0) + 1

probability_dict = {}
for i, j in count_dict.items():
    probability_dict[i] = j / total

interval_dict = {}
low = 0
for i in sorted(probability_dict):
    high = low + probability_dict[i]
    interval_dict[i] = (low, high)
    low = high

low = 0
high = 1

for i in data_lst:
    l, h = interval_dict[i]
    r = high - low
    high = low + r*h
    low = low + r*l

encoded_value = (low + high)/2
print(encoded_value)

```